# Workbook 1 Lesson 1

Workbook 01 • Foundations
Lesson 01 — Access Check (Reps → Solve → Refactor)

What you need before starting
You will work in VS Code using this repo folder structure:

`workbooks/wb01_foundations/lesson01/`

- `rep_page.py` (Page 1)
- `solve_page.py` (Page 2)
- `refactor_page.py` (Page 3)

`workbooks/wb01_foundations/tests/`

- `test_wb01_l01_p1_reps.py`
- `test_wb01_l01_p2_solve.py`
- `test_wb01_l01_p3_refactor.py`

You will run pytest for ONE page at a time so you always get a clean pass/fail.

Turn off suggestions in VS Code (required)
In VS Code:

- Settings → search “Inline Suggest” → disable “Editor: Inline Suggest Enabled”
- Settings → search “Suggest On Trigger Characters” → disable it
- Optional: disable “Accept Suggestion On Enter” (set to Off)

Page 1 — REPS (type the function 10 times)
File: `workbooks/wb01_foundations/lesson01/rep_page.py`

Goal:
Build muscle memory for:

- `def` + indentation
- cleaning input with `.strip()` and `.upper()`
- `if/else`
- returning exact strings

Instructions:

1. Open `rep_page.py`.

2. Type the function below EXACTLY as shown. No copy/paste.
3. Do 10 reps:
   - Delete the function completely and re-type it from scratch.
   - Save the file after each rep.

Type exactly this code:

```python
def access_status(badge_code: str) -> str:
    badge = badge_code.strip().upper()

    if badge == "OVERCLOCKED":
        return "ACCESS GRANTED"
    else:
        return "ACCESS DENIED"
```

After rep 10, run Page 1 tests (only):
In the VS Code terminal:

```
pytest -q workbooks/wb01_foundations/tests/test_wb01_l01_p1_reps.py
```

Done looks like:

- You see 2 passing tests (green / “passed”).

If it fails:

- Fix spelling, quotes, indentation, and the exact output strings.
- Re-run until it passes before moving on.

---

## Page 2 — SOLVE (write an interactive function using input)

File: `workbooks/wb01_foundations/lesson01/solve_page.py`

Goal:
Use real user input and connect it to the Page 1 logic.

Task:
Write a function:

`prompt_access() -> str`

Requirements (all required):

1. Ask the user for a badge code using `input()`
2. Pass the user's input into `access_status(...)` from Page 1
3. Print the returned status string
4. Return that same status string

Important:

- `access_status` returns the message. `prompt_access` prints it and returns it.
- Do NOT hardcode the badge code. It must come from `input()`.

Example behavior:

- If the user types `overclocked` the printed result should be `ACCESS GRANTED`
- If the user types `nope` the printed result should be `ACCESS DENIED`

Run Page 2 tests (only):

```
pytest -q workbooks/wb01_foundations/tests/test_wb01_l01_p2_solve.py
```

Done looks like:

- You see 1 passing test.

If it fails:

- You probably didn't print the result, didn't return it, or didn't call `input()`.

## Getting Help From AI

This part of the lesson is probably the most difficult, and the most important.
This is like the word problems in math class. Anyone can memorize the pattern for adding two numbers, but real problem solvers know that the information is never delivered to you like that.
In todays modern AI driven industry you will be expected to use AI as a tool to get work done, but be careful that you don't let it become a crippling factor in your learning.
The following is a prompt that you can paste into an AI platform of your choice that will have it help you work through the problem sets in a way that will help you solve the problem rather than just give you the solution.

```
— AI Help Prompt (Problem Decomposition, no spoilers)

Copy/paste this into an AI assistant before you start coding Page 2. Replace
nothing unless you want to add notes.
```

PROMPT:
You are my Python coach. Your job is to help me solve this without giving me the final code.

Rules you must follow:

- Do NOT write the full solution or a complete function for me.

- Do NOT give me a final code block that I can paste and run.

- You MAY show tiny fragments (1–3 lines) only when necessary to explain syntax.

- You MUST guide me using a problem decomposition process: Inputs → Outputs → Steps → Edge cases → Tests → Implementation plan.

- Ask me to type my attempt, then review it and point out exactly what to fix.

- If I'm stuck, give me a hint in increasing levels: Hint 1 (small), Hint 2 (bigger), Hint 3 (almost there).

- Keep focus on the specific task; no extra features.

My task :
I need to write a function named prompt_access() -> str in solve_page.py.

Requirements:

1. Ask the user for a badge code using input()

2. Pass that string into access_status(...) (from rep_page.py)

3. Print the returned status string

4. Return that same status string

Known behavior of access_status:

- It returns "ACCESS GRANTED" when the cleaned badge code equals "OVERCLOCKED"

- Otherwise it returns "ACCESS DENIED"

```
My test command:
pytest -q workbooks/wb01_foundations/tests/test_wb01_l01_p2_solve.py

Start by walking me through:

1. What are the inputs?

2. What are the outputs?

3. What are the exact steps in order?

4. What could go wrong?
    Then ask me to propose pseudocode, and only after that ask me to write the function.
```

---

## Page 3 — REFACTOR (same behavior, cleaner code)

File: `workbooks/wb01_foundations/lesson01/refactor_page.py`

Goal:
Learn the skill that makes real scripts look “advanced”: refactoring.

What “refactor” means:
You change how the code is written without changing what it does.

- Same inputs → same outputs
- You are improving clarity and structure, not adding features

What “pythonic” means (for this course):
Pythonic code is code that is:

- clear and readable
- simple (not clever)
- uses common Python patterns
  Pythonic is NOT “shortest possible code.”

Refactor target:
Rewrite `access_status(badge_code: str) -> str` so it behaves exactly like Page 1, but cleaner.

How to refactor this function (simple method):

1. Keep the sanitation line: `badge_code.strip().upper()`
2. Use an early return so you don't need `else` after a return
3. Keep the exact output strings unchanged

Rules:

- Must sanitize input (strip + upper)
- Must return EXACTLY the same strings as Page 1
- No printing inside `access_status` (printing belongs on Page 2)

## Getting Help From AI

For this part of lesson 1 you might not be able to figure out how to refactor your code. That's ok! The whole point of learning to code is to get a job, and part of that is solving problems. In fact problem solving is the most important thing you will lean here. In today's AI saturated industry, problem solving is the one skill that hiring managers are going to hire you for!
AI is a tool that you can (and in some cases will be expected to) use. But don't let it be a crutch for too long, because that will cripple your leaning and weaken your skill.
If you are having difficulty with the refactoring exercises, then use an AI to show you how to do it for the firs couple of lessons, but stop there. Don't let it become the norm. Be sure that you are paying attention to the output to lean the pattern and then type the code at least 10x to get it solid in your mind.

Remember, the reps and the struggle are how you learn.

```
 — AI Help Prompt (Refactor guidance, no spoilers)

Copy/paste this into an AI assistant before you start Page 3.

PROMPT:
You are my Python refactoring coach. Help me refactor without giving me the
finished code.

Rules you must follow:

- Do NOT output the final refactored function as complete code.

- Do NOT provide a paste-ready solution block.

- You MUST teach refactoring as a process: Behavior lock → Identify smells →
Apply one improvement → Re-run tests.

- You MAY provide small snippets (1-3 lines) to demonstrate a pattern (like
early return), but not the full function.
```

```
- You MUST make me explain what stays the same (inputs/outputs) before I
change anything.

- You MUST ask me to write my refactor attempt, then critique it and help me
fix it.

- Use “why” explanations briefly, but keep me moving.


My task:
In refactor_page.py, rewrite access_status(badge_code: str) -> str so it
behaves exactly like the rep_page.py version.

Behavior that must not change:

- It must sanitize input using strip() and upper()

- It must return EXACTLY "ACCESS GRANTED" or "ACCESS DENIED"

- Same inputs must produce same outputs as Page 1


Goal of the refactor:

- Make it more readable/pythonic (not shorter at all costs)

- Specifically, avoid an unnecessary else after a return (early return)


My test command:
pytest -q workbooks/wb01_foundations/tests/test_wb01_l01_p3_refactor.py

Start by making me do this checklist:

1. Write down the function’s contract (input/output)

2. List 3 example inputs and their outputs

3. Identify what part is “sanitation” and what part is “decision”

4. Describe one refactor move (early return) in plain English
    Then ask me to write my refactor attempt and paste it for review.
```

Run Page 3 tests (only):

```
pytest -q workbooks/wb01_foundations/tests/test_wb01_l01_p3_refactor.py
```

Done looks like:

- You see 1 passing test.

End of Lesson Check (optional)
After all three pages pass individually, you can run all tests:

```
pytest -q
```

If all pass, Lesson 01 is complete.